



Lecture 13: Iterating with Each Blocks

INTERVIEW QUESTION

[> CORE](#)

How do you render a list of items from an array, and why does each need a key?

You have ten users and want a row for each. When you sort the list, the rows lose their state and input values jump around. What went wrong?

Most web applications display lists of data: a feed of social media posts, a grid of products, or a table of users.

The Identity Crisis

Imagine you are building a banking dashboard with a list of "Pending Transactions". Each transaction row has a text input where the user can type a memo before approving it. The user types "Rent" into the first input, then clicks a button to sort the transactions by date.

Suddenly, the "Rent" note jumps to a completely different transaction, attaching itself to a \$5 coffee bill! What went wrong?

Without strict instructions, a rendering engine takes the path of least resistance. When the array sorted, it didn't actually move the HTML rows around. Instead, it left the HTML elements exactly where they were, and just updated the text inside them to match the new array order. But the text you typed into the `<input>` wasn't part of the array data; it was internal DOM state. Because the row itself never moved, the input text stayed at the top of the list, while the transaction data shifted beneath it.

The `{#each}` Block

To render a list of items in Svelte, you use the `{#each}` block. It takes an array and loops through it, rendering a block of HTML for every item.

To prevent the catastrophic identity crisis described above, Svelte allows you to define a **key** by providing a unique identifier in parentheses at the end of the declaration.

```
Transactions.svelte
1  <script>
2    let transactions = $state([
3      { id: 'tx_1', amount: 50, merchant: 'Coffee Shop' },
4      { id: 'tx_2', amount: 1200, merchant: 'Landlord' }
5    ]);
6  </script>
7
8  {#each transactions as tx (tx.id)}
   The (tx.id) is the unique key mapping data to the DOM
9    <div class="row">
10      <p>{tx.merchant}: ${tx.amount}</p>
11      <input placeholder="Add memo..." />
12    </div>
13  {/each}
```

Why Keys are Mandatory

When you provide `(tx.id)` as the key, you are making a binding contract with the Svelte compiler. You are telling it: *"This specific piece of data is permanently married to this specific HTML `<div>`."*

Now, when the array is sorted, Svelte will not cheat by just updating the text. It will grab the exact DOM node tied to `tx_2` and physically move it down the page. Because the entire HTML element moves, any internal state (like the text the user typed into the memo input, or CSS animations currently playing on that node) moves perfectly with it.

KEY TAKEAWAY

Always use a unique, stable database ID for your key.
Never use the array index `( index )` as a key if the list can be reordered, because the index of an item changes when it moves, defeating the entire purpose of the key.

Summary

To render arrays of data, wrap your markup in an `{#each array as item}` logic block. To ensure the DOM updates safely and predictably when the array changes, you must provide a unique identifier for every item using the `(key)` syntax. This key locks the data to a specific DOM node, ensuring that sorting, filtering, or deleting items physically moves the correct HTML elements, preserving any internal DOM state like focus or user input.

	NO KEY (Unsafe)	WITH KEY (Safe)
Syntax	<code>{#each items as item}</code>	<code>{#each items as item (item.id)}</code>
Sorting Behavior	Updates text in place	Physically moves DOM nodes
DOM State (Inputs)	Corrupts and jumps around	Moves safely with the item